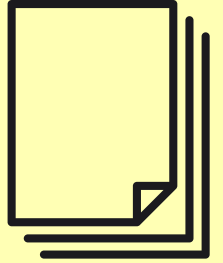


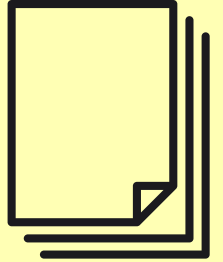
Starter: Symmetric Encryption

Decode:

ALEX MW XLMW XCTI SJ IRGVCTXMSR GEPPIH



Starter: Symmetric Encryption



Decode:

ALEX MW XLMW XCTI SJ IRGVCTXMSR GEPPIH

WHAT IS THIS TYPE OF ENCRYPTION CALLED

Substitution Cipher → Caesar Cipher, key 4

Definitions



What is encryption?

What is symmetric encryption?

Definitions



What is encryption?

A procedure used to convert plaintext into ciphertext in order to prevent any but the intended recipient from reading that data.

What is symmetric encryption?

A type of encryption that uses the same key for both encryption of plaintext and decryption of ciphertext.

Vernam Cipher



A sensitive message could be encrypted by using either the Vernam Cipher or the Caesar Cipher.

1. Explain why the Vernam cipher may be a better choice. (2 points)
2. What four conditions must be met in order for the Vernam cipher to offer perfect security?

Vernam Cipher



1. Vernam cipher (if implemented correctly) is unbreakable / harder to crack / Caesar cipher can be easily cracked;
Frequency / statistical analysis of ciphertext reveals nothing about plaintext;
More possible keys;
Vernam cipher does not always translate a ciphertext character to the same plaintext character (removing repeated patterns);

2. The key must be (at least) as long as the data to be encrypted/
plaintext;

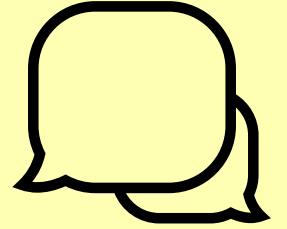
The key must not be reused // key must only be used once;

The key must be (truly) random;

The key must be kept securely/not revealed/only known by user(s);

Symmetric Encryption

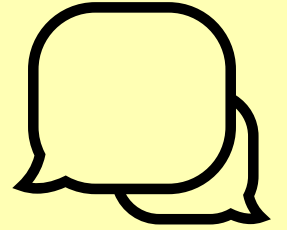
In pairs discuss what are the main problems with symmetric encryption?



Symmetric Encryption

In pairs discuss what are the main problems with symmetric encryption?

Hard to distribute the key without interception.
Anyone with the key can decrypt.



Symmetric Encryption, the problem.

If we were to use Symmetric encryption, you would need to have a publically visible key.

That would be used to decrypt anything you send.

That could also be used to encrypt something if I wished to **pretend** to be you...

Asymmetric Encryption

In the 1970s, it was realised that it is possible to have an asymmetric key pair (ie different key for encryption and decryption), solving the key-distribution issue.

Asymmetric Encryption

In the 1970s, it was realised that it is possible to have an asymmetric key pair (ie different key for encryption and decryption), solving the key-distribution issue.

As long as the decryption key cannot be found from the encryption key, then the message is 'safe'.

Asymmetric Encryption

In the 1970s, it was realised that it is possible to have an asymmetric key pair (ie different key for encryption and decryption), solving the key-distribution issue.

As long as the decryption key cannot be found from the encryption key, then the message is 'safe'.

To do this, you need to use a **pair of keys**, one public and one private key.

Asymmetric Encryption

In the 1970s, it was realised that it is possible to have an asymmetric key pair (ie different key for encryption and decryption), solving the key-distribution issue.

As long as the decryption key cannot be found from the encryption key, then the message is 'safe'.

To do this, you need to use a **pair of keys**, one public and one private key.

Asymmetric Encryption, Digital Signatures & Digital Certificates

In order to encrypt for transmission, we need to know what everyone has.



 **Public Key**

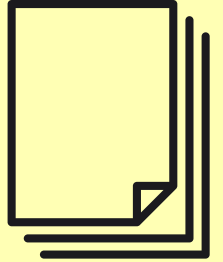
 **Private Key**



 **Public Key**

 **Private Key**

Encrypting a document

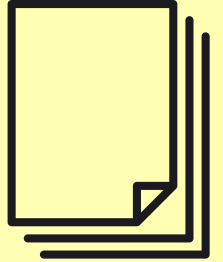


When one person wants to send a document to another person, they need to use two keys. One public, one private.

To ensure their intended recipient can actually decrypt the message, they will use:

- Their own **private key** 🔑
- The recipient's **public key** 🔑

Encrypting a document



When one person wants to send a document to another person, they need to use two keys. One public, one private.

To ensure their intended recipient can actually decrypt the message, they will use:

- Their own **private key** 🔑
- The recipient's **public key** 🔑

What do you think is used to decrypt the message?

The RSA Method

Note, this is beyond spec.
It's just really interesting.

Practically, generating these numbers is complicated, but a method was created by Rivest, Shamir & Adelman.

1. Take 2 prime numbers, p , q (large enough...)

The RSA Method

Note, this is beyond spec.
It's just really interesting.

Practically, generating these numbers is complicated, but a method was created by Rivest, Shamir & Adelman.

1. Take 2 prime numbers, p, q (large enough...)
2. Multiply together, $n = pq$

The RSA Method

Note, this is beyond spec.
It's just really interesting.

Practically, generating these numbers is complicated, but a method was created by Rivest, Shamir & Adelman.

1. Take 2 prime numbers, p, q (large enough...)
2. Multiply together, $n = pq$
3. Find 2 related numbers, a, b

The RSA Method

Note, this is beyond spec.
It's just really interesting.

Practically, generating these numbers is complicated, but a method was created by Rivest, Shamir & Adelman.

1. Take 2 prime numbers, p, q (large enough...)
2. Multiply together, $n = pq$
3. Find 2 related numbers, a, b
4. n, a is your **public key**

The RSA Method

Note, this is beyond spec.
It's just really interesting.

Practically, generating these numbers is complicated, but a method was created by Rivest, Shamir & Adelman.

1. Take 2 prime numbers, p, q (large enough...)
2. Multiply together, $n = pq$
3. Find 2 related numbers, a, b
4. n, a is your **public key**
5. n, b is your **private key**

The RSA Method

Note, this is beyond spec.
It's just really interesting.

Practically, generating these numbers is complicated, but a method was created by Rivest, Shamir & Adelman.

1. Take 2 prime numbers, p, q (large enough...)
2. Multiply together, $n = pq$
3. Find 2 related numbers, a, b
4. n, a is your **public key**
5. n, b is your **private key**

Can the private key be discovered from knowing the public key?

The RSA Method

Note, this is beyond spec.
It's just really interesting.

Practically, generating these numbers is complicated, but a method was created by Rivest, Shamir & Adelman.

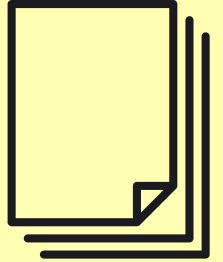
1. Take 2 prime numbers, p, q (large enough...)
2. Multiply together, $n = pq$
3. Find 2 related numbers, a, b
4. n, a is your **public key**
5. n, b is your **private key**

Can the private key be discovered from knowing the public key?

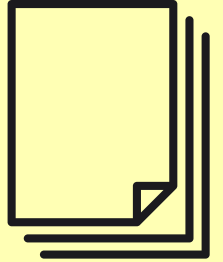
Yes, if we can factorise n , which is believed to be hard for large numbers.

Digital Signatures

With a private key and the right software, you can put a digital signature onto documents after the data.



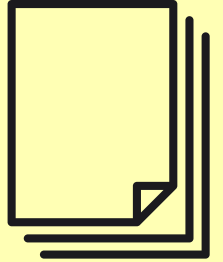
Digital Signatures



With a private key and the right software, you can put a digital signature onto documents after the data.

A digital signature is a ‘stamp’ placed on the data which is unique to you, and very hard to forge.

Digital Signatures



With a private key and the right software, you can put a digital signature onto documents after the data.

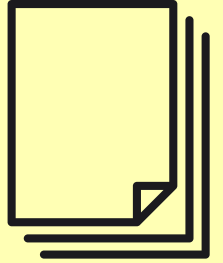
A digital signature is a ‘stamp’ placed on the data which is unique to you, and very hard to forge.

In addition, the signature assures that changes made to the data cannot go undetected.

The process:

Creating the Signature

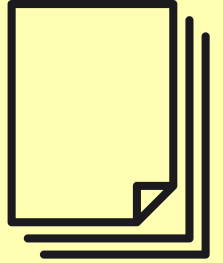
1. The entire message to be sent is **hashed**.



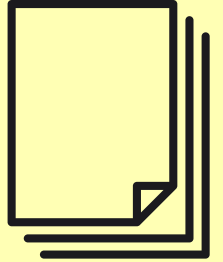
The process:

Creating the Signature

1. The entire message to be sent is **hashed**.
2. This hash is then encrypted using your **private key**



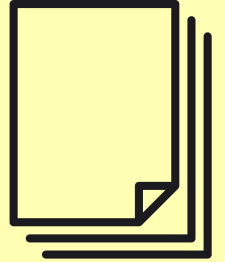
The process:



Creating the Signature

1. The entire message to be sent is **hashed**.
2. This hash is then encrypted using your **private key**
3. The result of this is the **Digital Signature**, which is appended to the original message.

The process:



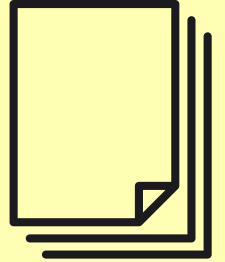
Creating the Signature

1. The entire message to be sent is **hashed**.
2. This hash is then encrypted using your **private key**
3. The result of this is the **Digital Signature**, which is appended to the original message.

Verifying the Signature

1. The contents of the message is hashed by the recipient.

The process:



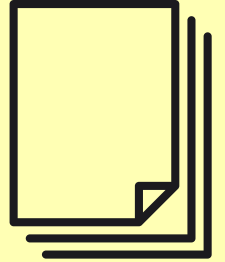
Creating the Signature

1. The entire message to be sent is **hashed**.
2. This hash is then encrypted using your **private key**
3. The result of this is the **Digital Signature**, which is appended to the original message.

Verifying the Signature

1. The contents of the message is hashed by the recipient.
2. The Digital signature is decrypted using your public key.

The process:



Creating the Signature

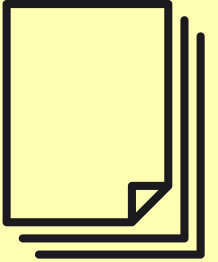
1. The entire message to be sent is **hashed**.
2. This hash is then encrypted using your **private key**
3. The result of this is the **Digital Signature**, which is appended to the original message.

Verifying the Signature

1. The contents of the message is hashed by the recipient.
2. The Digital signature is decrypted using your public key.
3. Both of these should produce the same result, verifying that no changes have been made.

A further complication...

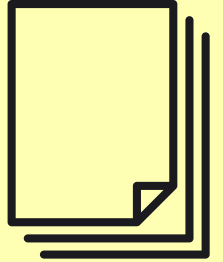
This all assumes that nobody is offering false public keys on behalf of someone else...



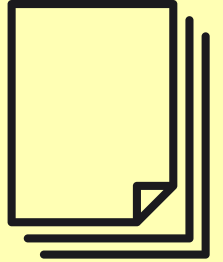
A further complication...

This all assumes that nobody is offering false public keys on behalf of someone else...

Enter a hero: **A Digital Certificate** (different to a signature)



A further complication...

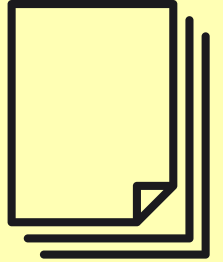


This all assumes that nobody is offering false public keys on behalf of someone else...

Enter a hero: **A Digital Certificate** (different to a signature)

A Digital Certificate can be made by a Certificate Authority Centre. It includes some information about you, as well as a copy of your public key.

A further complication...



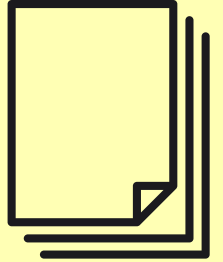
This all assumes that nobody is offering false public keys on behalf of someone else...

Enter a hero: **A Digital Certificate** (different to a signature)

A Digital Certificate can be made by a Certificate Authority Centre. It includes some information about you, as well as a copy of your public key.

When a message comes in, you can request the Certificate from the Certificate Authority, rather than anyone else.

A further complication...



This all assumes that nobody is offering false public keys on behalf of someone else...

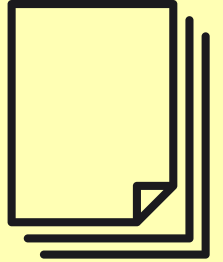
Enter a hero: **A Digital Certificate** (different to a signature)

A Digital Certificate can be made by a Certificate Authority Centre. It includes some information about you, as well as a copy of your public key.

When a message comes in, you can request the Certificate from the Certificate Authority, rather than anyone else.

But what if someone alters something sent by the Certificate Authority?

A further complication...



This all assumes that nobody is offering false public keys on behalf of someone else...

Enter a hero: **A Digital Certificate** (different to a signature)

A Digital Certificate can be made by a Certificate Authority Centre. It includes some information about you, as well as a copy of your public key.

When a message comes in, you can request the Certificate from the Certificate Authority, rather than anyone else.

But what if someone alters something sent by the Certificate Authority?
Another **Digital Signature** check...

That's a lot to do, right?

Well, yes.

Thankfully, all of this is handled behind the scenes by security software, which normally is either hidden (like inside .pdf s), or requires little other than a few clicks.

Hybrid Cryptosystems

Again, that is a lot of work.

Hybrid Cryptosystems

Again, that is a lot of work.

Generally, we don't use asymmetric encryption for everything.

Hybrid Cryptosystems

Again, that is a lot of work.

Generally, we don't use asymmetric encryption for everything.

Symmetric encryption is both easier and faster.

Hybrid Cryptosystems

Again, that is a lot of work.

Generally, we don't use asymmetric encryption for everything.

Symmetric encryption is both easier and faster.

We can use asymmetric encryption initially to solve symmetric encryption's largest flaw: **secure key sharing**

Hybrid Cryptosystems

Again, that is a lot of work.

Generally, we don't use asymmetric encryption for everything.

Symmetric encryption is both easier and faster.

We can use asymmetric encryption initially to solve symmetric encryption's largest flaw: **secure key sharing**

This is what PGP, SSH and the SSL/TLS family of schemes use. They are thus called **hybrid cryptosystems**

Practice Time!

L236 - Asymmetric Encryption Worksheet

When you're done: **2023 P2 Q11.5**

